
CSE D 501

Kernels cont'd, Clustering
Taylor Kessler Faulkner
University of Washington
October 29, 2025



Today's Class

- Kernels continued
- Break
- Intro to clustering, K-means, GMMs
- Break
- Hierarchical clustering

Nonlinear decision boundaries and kernels

- Original data in 1 dimension: not linearly separable
- Introduce a new feature through a **mapping**: $\phi(x) = \begin{bmatrix} x_1 \\ x_1^2 \end{bmatrix}$
- Now we can find a linear separation in 2 dimensions!

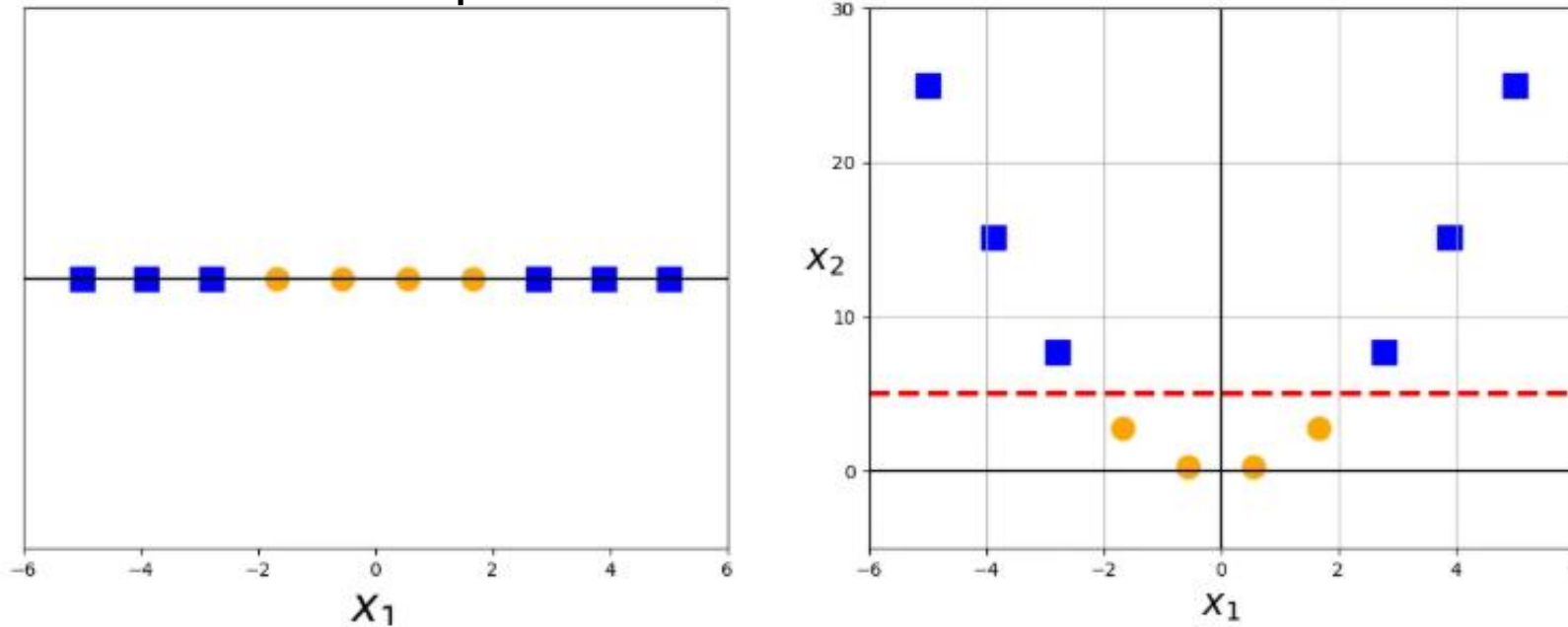


Image from <https://medium.com/data-science/the-kernel-trick-c98cdbcaeb3f>

Changing the topic: nonlinear data and kernels

- Example in 2d: $\phi(x) = x \bmod 2$

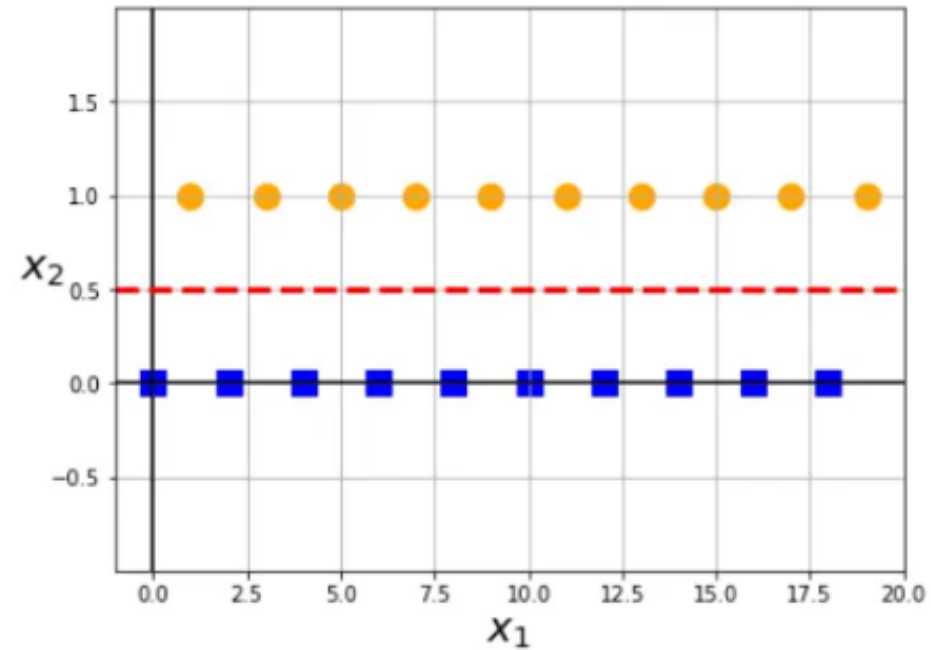
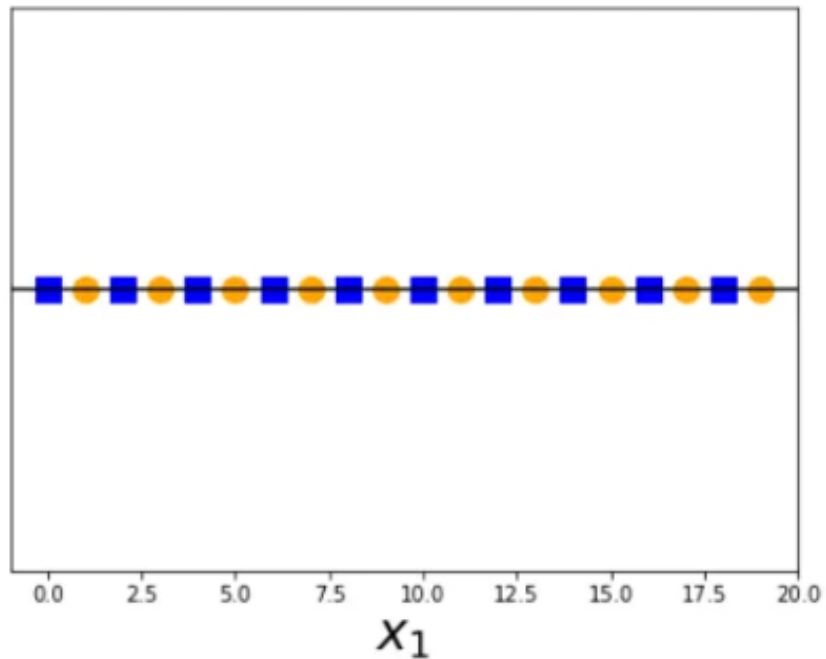


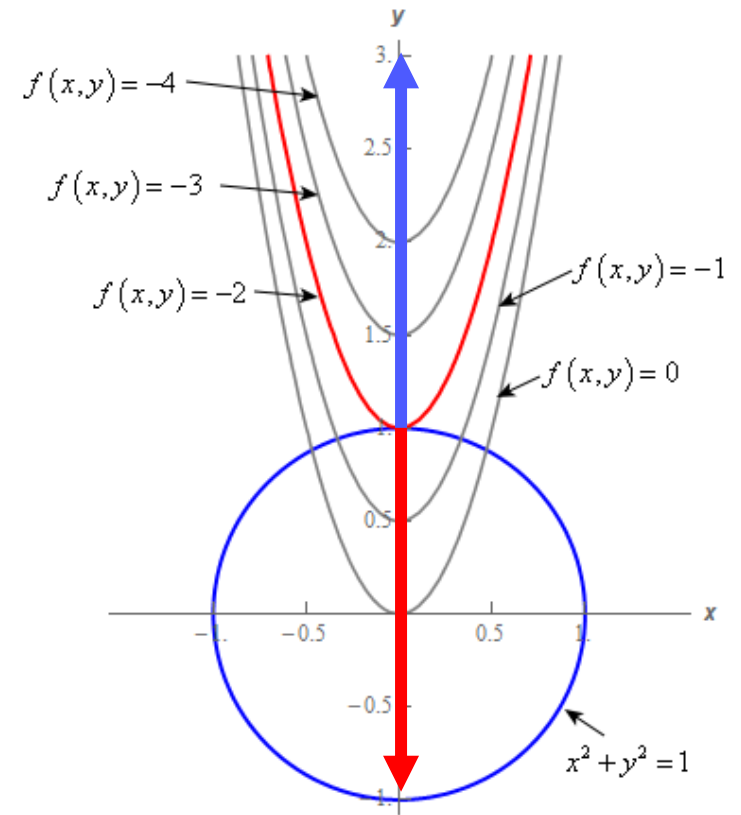
Image from <https://medium.com/data-science/the-kernel-trick-c98cdbcaeb3f>

Nonlinear data and kernels

- When talking about SVMs, we mentioned that they can also be used for nonlinear data
- This can be done by using the idea that non-linearly separable data can be linearly separable in higher dimensions
- But first, we have to reformulate the SVM solution
- (Note: this involves Calc III; we'll walk through the intuition of why the reformulation works, but the main thing to take away is what a kernel is and why we use them)

Dual formulation for SVM

- We're going to use **Lagrange multipliers** to reformulate our SVM
- Basic idea behind Lagrange multipliers:
 - You can turn a constrained optimization problem into an unconstrained optimization problem
 - And remember, our SVM solution is a constrained optimization problem!
 - $\min_{w,b} \frac{1}{2} \|w\|_2^2$ subject to constraints: $y_i(\mathbf{w}^T x_i + b) \geq 1$
 - Assuming the two classes are -1 and 1
- Lagrange multipliers: when your optimizing function is minimized with constraints, the gradient of the optimizing function is **parallel** to the gradient of the constraint function



Dual formulation for SVM

- If your optimizing function is $f(x)$, and your constraint function is $g(x) = 0$ the fact that their gradients are parallel means that

$$\nabla f = \alpha \nabla g$$

- We call α here a **Lagrange multiplier**
- To find the x that minimizes $f(x)$ with constraints $g(x)$, we can find

$$\min_x (\max_{\alpha} (f(x) - \alpha g(x)))$$

- Intuition behind why we can use this new function
 - $\max_{\alpha} (f(x) - \alpha g(x)) = f(x)$ when $g(x) = 0$
 - $\max_{\alpha} (f(x) - \alpha g(x)) = \infty$ when $g(x) \neq 0$
 - So this lets us find the minimum of $f(x)$ subject to our original constraint

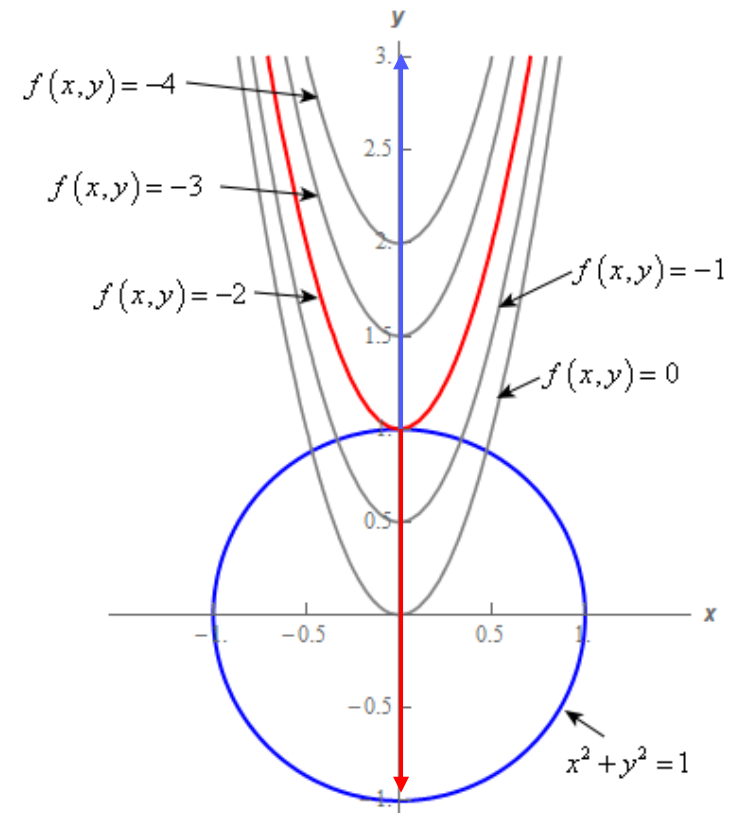


Image from
<https://tutorial.math.lamar.edu/classes/calciil/lagrangemultipliers.aspx>

Dual formulation for SVM

- Translating this into the SVM:
 - $\min_{w,b} \frac{1}{2} \|w\|_2^2$ subject to constraints: $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$
 - Goes to $L(w, b, \alpha) = \min_{w,b} \max_{\alpha} \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i [(\mathbf{w} \cdot \mathbf{x}_i + b)y_i - 1]$
 - $\mathbf{w} \cdot \mathbf{x}_i$: dot product, equivalent here to $\mathbf{w}^T \mathbf{x}_i$
 - We can reformulate as $\max_{\alpha} \min_{w,b} \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i [(\mathbf{w} \cdot \mathbf{x}_i + b)y_i - 1]$
 - Flip min and max
- To obtain the minimum with respect to w and b , we must find where the partial derivative with respect to w and b are 0
 - $\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0$
 - $w = \sum_{i=1}^n \alpha_i y_i x_i$
 - $\frac{\partial L}{\partial b} = -\sum_{i=1}^n \alpha_i y_i = 0$
 - $\sum_{i=1}^n \alpha_i y_i = 0$

Dual formulation for SVM

- We'll skip ahead a few steps for time, but now that we've rewritten w and b , we can find the following formulation of our original problem:

$$\max_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i$$

- Once you solve this **dual formulation**, the decision for a new input x is given by the sign of:

$$\sum_{i=1}^n \alpha_i y_i (x_i \cdot x) + b$$

- b can be found by using any support vector x_i , which all satisfy

$$y_i (w^T x_i - b) = 1$$

- If we're working with high-dimensional data to try to transform into a linearly-divisible space, we can use a map $\phi(x)$ instead of x in these equations
- But computing and storing high-dimensional data and then computing dot products is computationally expensive!

Kernel Trick

- Ok, so why did we rewrite this problem in the first place?
- We're going to use something called the **kernel trick**
- $K(x, x'): \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a **kernel** for a map $\phi(x)$ if $K(x, x') = \phi(x) \cdot \phi(x')$
- If we can represent models as functions of dot products of feature maps, and we can find a kernel for that feature map such that $K(x, x') = \phi(x)^T \phi(x')$, we can avoid explicitly storing and computing high-dimensional maps!

Kernel Trick

- Now we can use kernels in our SVM for nonlinear data!
- Instead of

$$\max_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i$$

- We'll use

$$\max_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) - \sum_{i=1}^n \alpha_i$$

- Where $K(x_i, x_j)$ computes the dot product of a mapping for x
- And the decision for a new input x can be found by the sign of

$$\sum_{i=1}^n \alpha_i y_i K(x_i, x) + b$$

Dual formulation for SVM

- Example:

$$\phi(x) = \begin{bmatrix} x[1]^2 \\ \sqrt{2}x[1]x[2] \\ x[2]^2 \end{bmatrix}, K(x, x') = (x^T x')^2$$

- Consider $x = [1, 2], x' = [3, 4]$
- Show that the output of the mapping $\phi(x)^T \phi(x')$ and the kernel are the same
- Reminder of $x^T x' = x[1]x'[1] + x[2]x'[2]$

Dual formulation for SVM

- Example:

$$\phi(x) = \begin{bmatrix} x[1]^2 \\ \sqrt{2}x[1]x[2] \\ x[2]^2 \end{bmatrix}, K(x, x') = (x^T x')^2$$

- Consider $x = [1, 2], x' = [3, 4]$

- $\phi(x)^T \phi(x) = [1, \sqrt{2} * 2, 4] \begin{bmatrix} 9 \\ \sqrt{2} * 12 \\ 16 \end{bmatrix} = 9 + 2 * 24 + 64 = 121$

- $K(x, x') = [1, 2] \begin{bmatrix} 3 \\ 4 \end{bmatrix}^2 = (3 + 8)^2 = 121$

- Kernels are *much* faster than explicitly computing maps!

Popular Kernels

- Polynomials of degree k
 - $K(x, x') = (x^T x')^k$
- Gaussian kernel (RBF kernel, Radial Basis Function)
 - $K(x, x') = \exp\left(-\frac{\|x - x'\|_2^2}{2\sigma^2}\right)$
 - σ is a hyperparameter for variance
 - Based on Gaussian distribution
- Sigmoid kernel
 - $K(x, x') = \tanh(\gamma x^T x' + r)$
 - γ is a hyperparameter for scaling
 - r is a hyperparameter for bias
 - Compare to sigmoid function for Logistic Regression

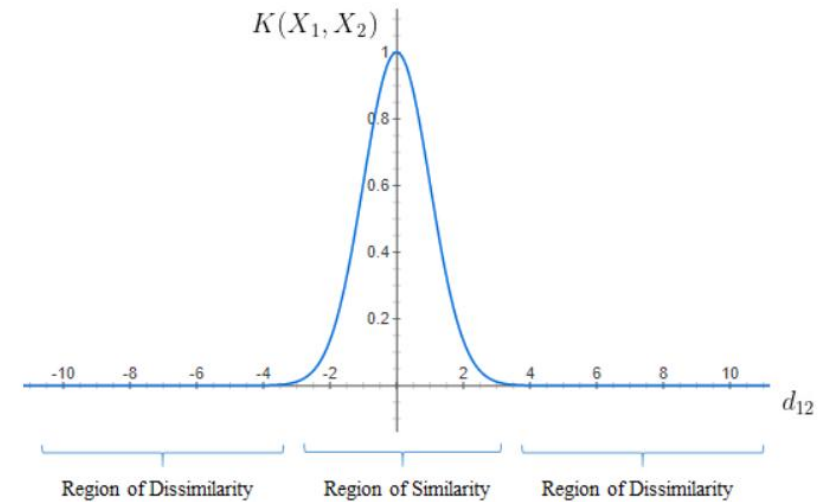


Image from
<https://towardsdatascience.com/radial-basis-function-rbf-kernel-the-go-to-kernel-acf0d22c798a>



Demo

Kernel PCA

- We discussed that PCA is great for dimensionality reduction when data is linear
- PCA can also be used with kernels for nonlinear data
- Example image: Linear PCA vs RBF kernel PCA

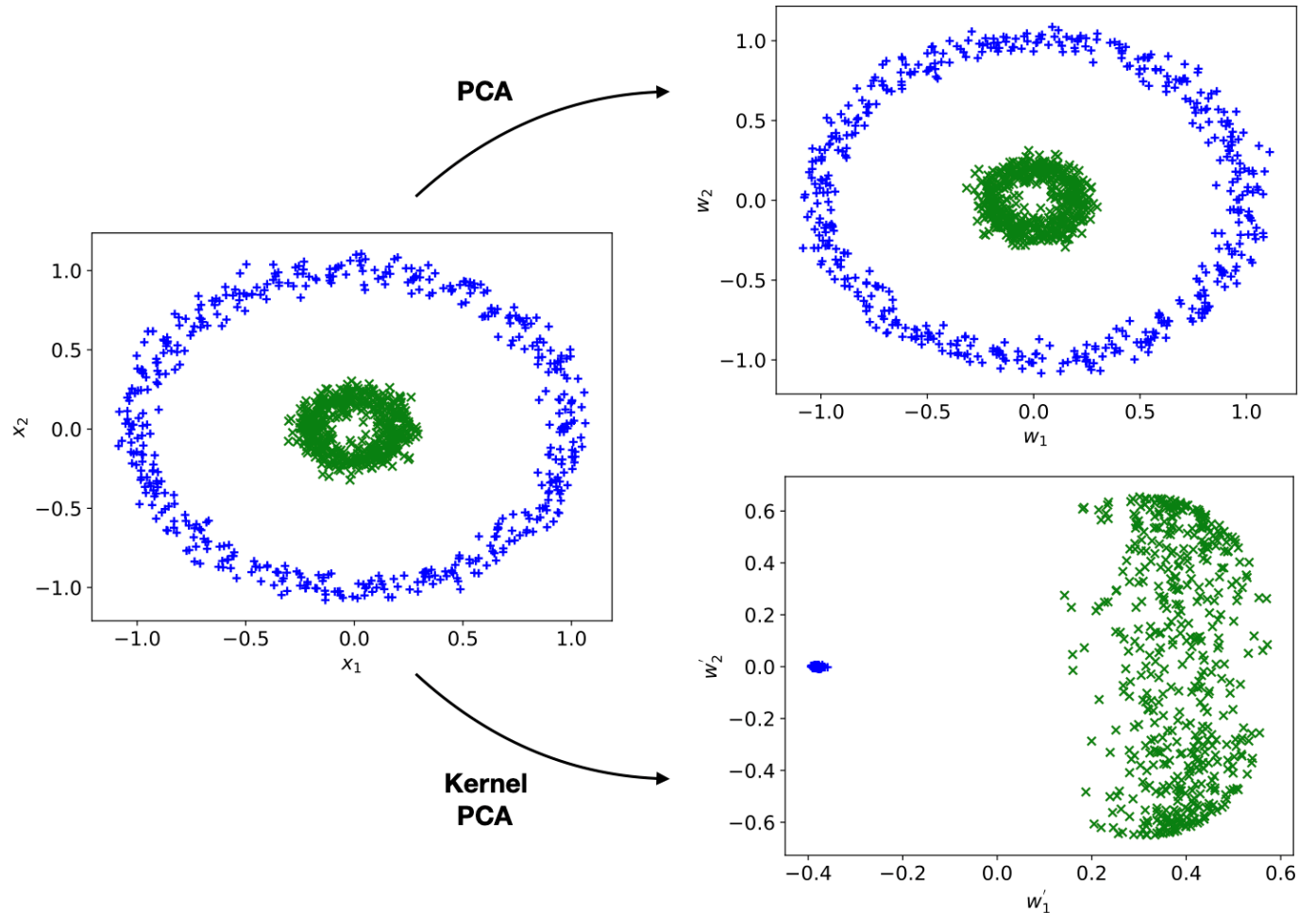


Image from https://ml-lectures.org/docs/structuring_data/ml_without_neural_network-2.html

PCA Algorithm

Input data: Matrix X , the $n \times D$ matrix containing your data and features

1. Center the data: Subtract the mean from each row

$$X_c = X - \bar{X}[0:D-1]$$

2. Compute covariance matrix Σ

$$\Sigma[t, s] = \frac{1}{n} \sum_{i=0}^{n-1} X_c[i, t] X_c[i, s] = \frac{X_c^T X_c}{n}$$

3. Find the eigenvectors and eigenvalues of Σ

1. Select k eigenvectors ($u_0, \dots, u_{k-1}$) with largest eigenvalues: these are your **principal components**

4. Project data: project the data onto the **principal components**

$$z_i[0] = u_0^T X_c[i] = u_0[1] X_c[i, 1] + \dots + u_0[D] X_c[i, D]$$

...

$$z_i[k-1] = u_{k-1}^T X_c[i] = u_{k-1}[1] X_c[i, 1] + \dots + u_{k-1}[D] X_c[i, D]$$

Kernel PCA

- Assuming N samples of original data:
 - Create an $N \times N$ **kernel matrix** K of the data, where
$$K[i, j] = K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$
 - Center the data (make all mapped features $\phi(x_j)$ have mean 0)
 - Plugging $\phi(x_i) = \phi(x_i) - \frac{1}{N} \sum_{k=1}^N \phi(x_k)$ into the definition of K and doing some algebra/matrix math we can find

$$\tilde{K} = K - 1_N K - K 1_N + 1_N K 1_N$$

where 1_N is an $N \times N$ matrix with all values equal to $\frac{1}{N}$

- Find the eigenvalues and eigenvectors of $\tilde{K}$, λ_j , α_j
- For any data point x , we can now represent it as the following set of features for $j = 1, \dots, D$ (dimensions):

$$z[j] = \sum_{i=1}^N \alpha_j[i] K(x, x_i)$$

Break

Get some water, stretch your legs, chat, be back in 10

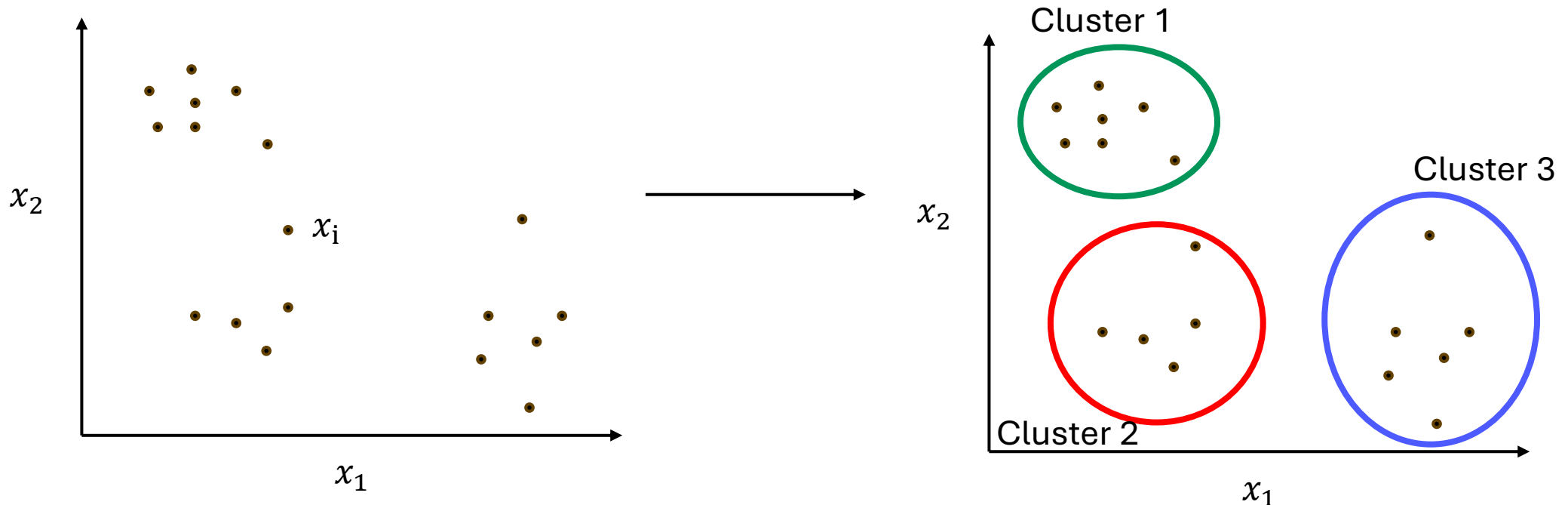
Unsupervised Learning

Unsupervised Learning

- We have mostly covered **supervised learning** algorithms up until now
 - One exception: PCA
- This class will introduce more **unsupervised learning** methods
 - No labels/outputs/“correct answers” for learning
- The following concepts no longer apply:
 - Error
 - Accuracy
 - Bias/variance tradeoff
 - Overfitting
- Examples of unsupervised learning tasks
 - Dimensionality reduction (PCA)
 - Finding groups of similar items for recommendations, e.g. “what to watch next” on streaming services
 - Grouping customers into subgroups for marketing and other analysis

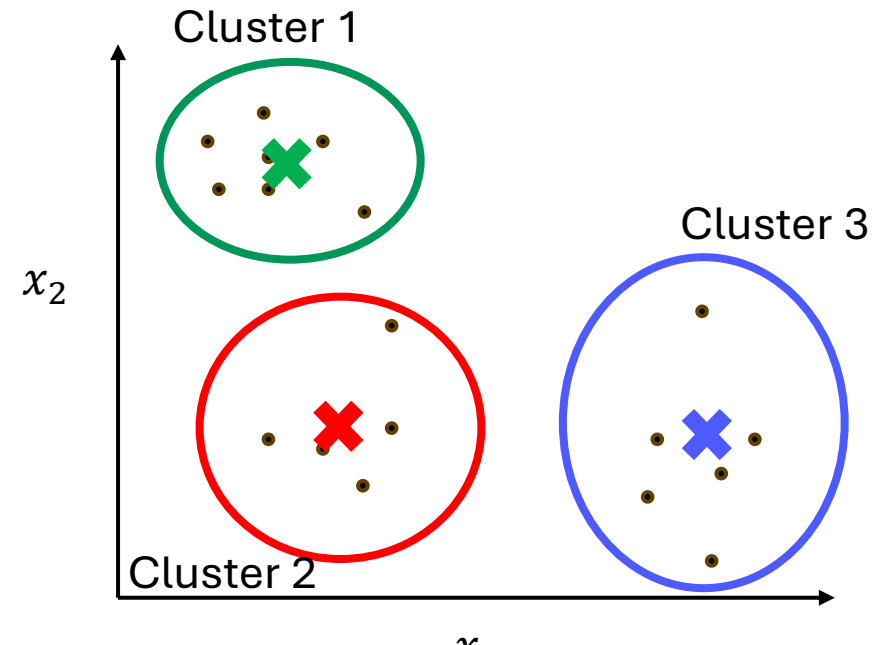
Clustering

- Often, real-world data does not come with labels
 - If you need to do classification/regression, you can label data yourself
 - Or, if you just want to uncover the underlying data structure, you can use **clustering**!
- **Clustering** is the ML process for finding related groups within an unlabeled dataset X
 - Sometimes, we may also want to cluster labeled data to examine its structure! In that case, we can remove the labels and perform clustering.



Clustering

- Base cluster definitions:
 - **Centroid:** the location and size of the cluster's center
 - **Spread:** the shape/size and density of a cluster
- Clustering algorithms:
 - Assign each datapoint x_i to a cluster
 - This assignment is based on the similarity of datapoints to other cluster points and/or centroids
- We'll start by focusing on **distance metrics**
- Different distance metrics can be used, just like in k-Nearest Neighbors
- Close distance means similar datapoints, far distance means dissimilar
- Standardize data to make sure different features are on the same scale!



K-means Clustering

- The k-means algorithm is one of the simpler clustering algorithms
- Algorithm:

Given a dataset of n datapoints and hyperparameter k

Step 0: Initialize cluster centroids randomly

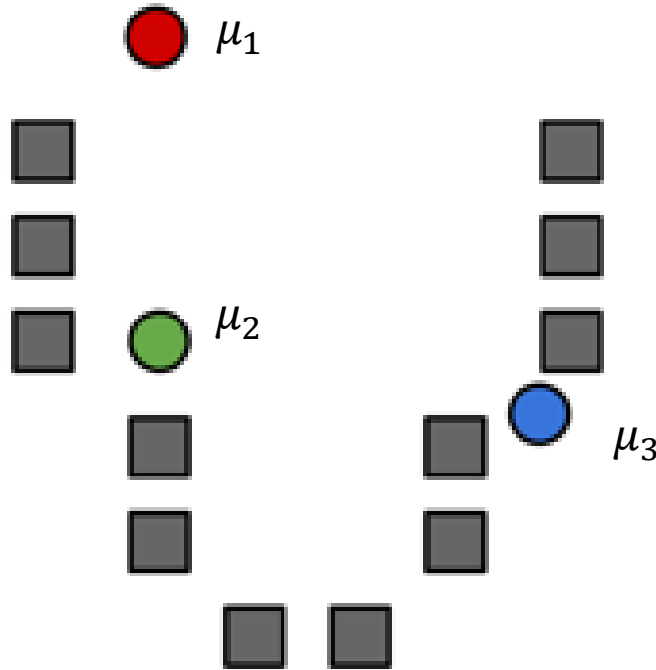
Repeat until convergence:

Step 1: Assign each example to its closest cluster centroid

Step 2: Update the centroids to be the average of all the points assigned to that cluster

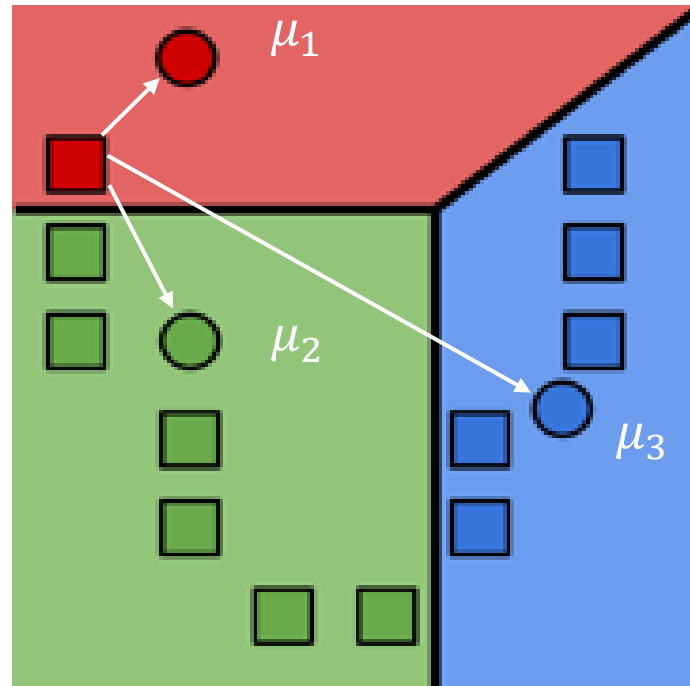
K-means Clustering: Step 0

- Choose the initial cluster centroids
- To begin, we'll use a common default choice, which is random initialization. Choose $\mu_1, \dots, \mu_k$ randomly



K-means Clustering: Step 1

- Assign each sample to its closest centroid (k-means uses Euclidean distance)
- For $i \in [1, \dots, n]$, $z_i \leftarrow \operatorname{argmin}_{j \in [1, \dots, k]} \operatorname{dist}(\mu_j, x_i)$
- Note: the coloring here is showing the Voronoi Tessellation, like in k-NN!

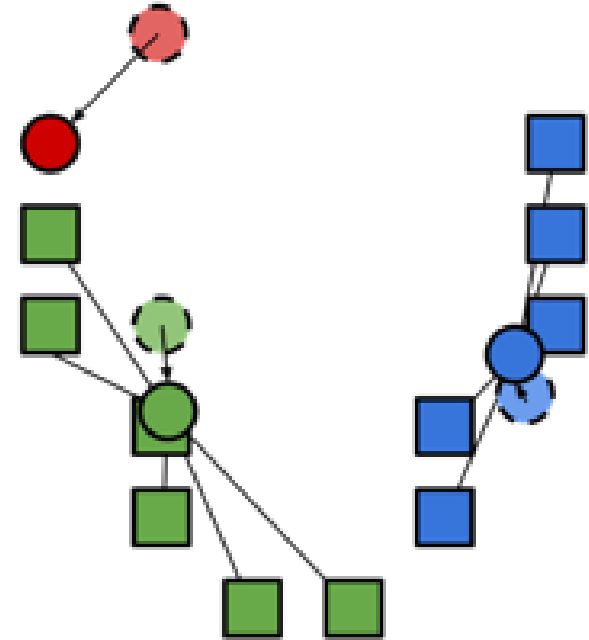


K-means Clustering: Step 2

- Update the centroids to be the mean of points assigned to that cluster
 - Equivalent to computing center of mass

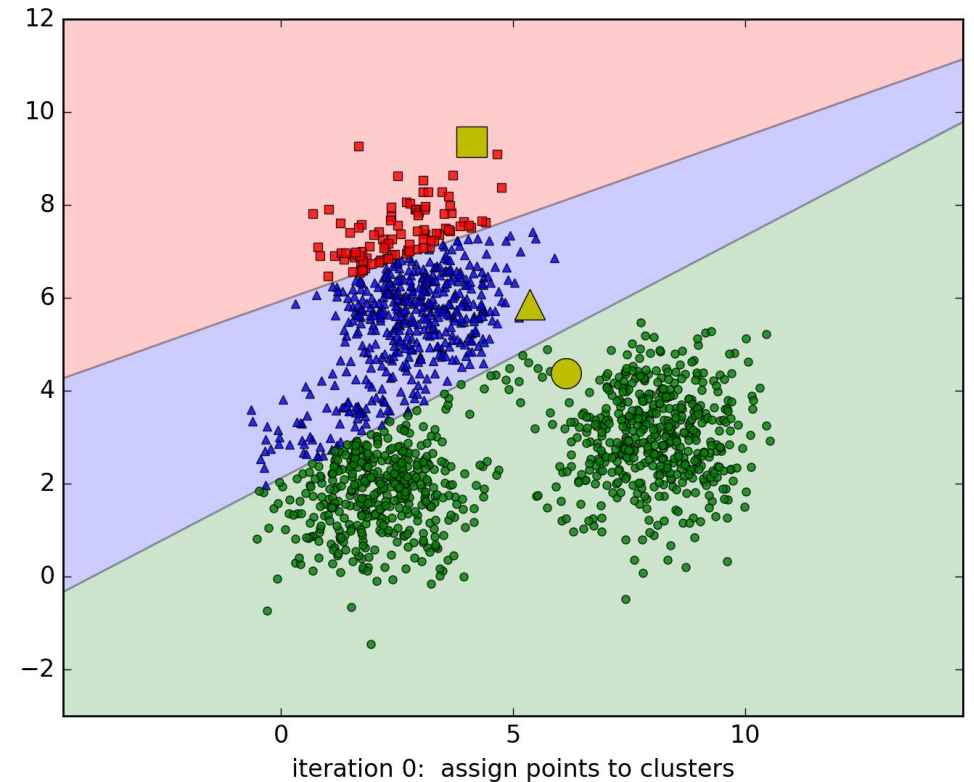
$$\mu_j = \frac{\sum_{i=1}^n \mathbf{1}\{z_i = j\} x_i}{\sum_{i=1}^n \mathbf{1}\{z_i = j\}}$$

- Numerator: sum of datapoints assigned to cluster j
- Denominator: number of datapoints assigned to cluster j



K-means Clustering: Convergence

- Repeat steps 1 and 2 until the algorithm converges
 - Datapoints do not change cluster assignments
- or
- Centroids do not change
- or
- A maximum number of iterations has been reached



K-means Clustering: Choosing k

- The hyperparameter k has a large effect on the outcome of k-means

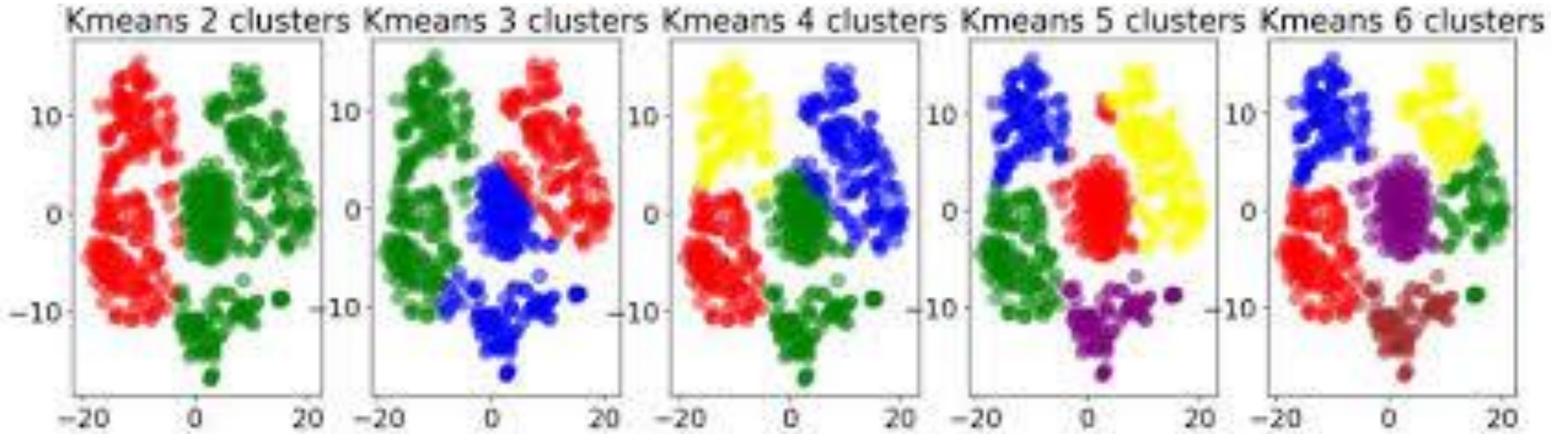


Image from Fang, Meng, et al. "Application of unsupervised clustering model based on graph embedding in water environment." *Scientific Reports* 13.1 (2023): 22774.

K-means Clustering: Choosing k

- We can use a metric WSS (within-cluster sum of squares) to measure how much the data points in each cluster vary from their centroid
- Formula: sum of the squared distances between each datapoint and the cluster centroid

$$WSS = \sum_{i=1}^k \sum_{j=1}^{n_i} \|x_j^{(i)} - \mu_i\|^2$$

- Where n_i is the number of datapoints in cluster i and $x_j^{(i)}$ is the j -th datapoint in the cluster i
- To find the optimal k , you can use domain knowledge (how many groups do you want to end up with?) or use an elbow plot of WSS for varying numbers of clusters

K-means Clustering: Choosing k

- Elbow plot reminder: find where the WSS forms an “elbow”

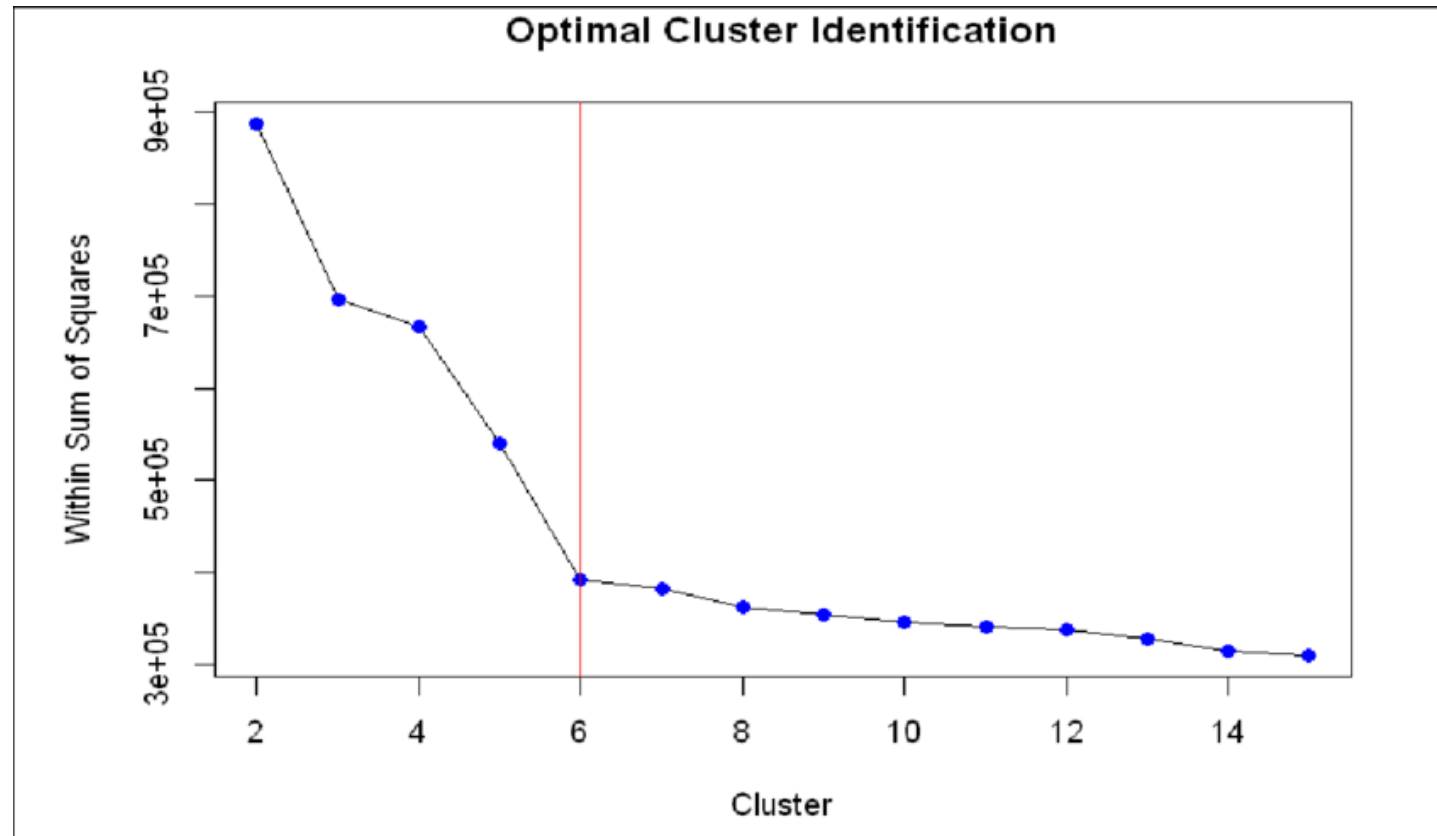


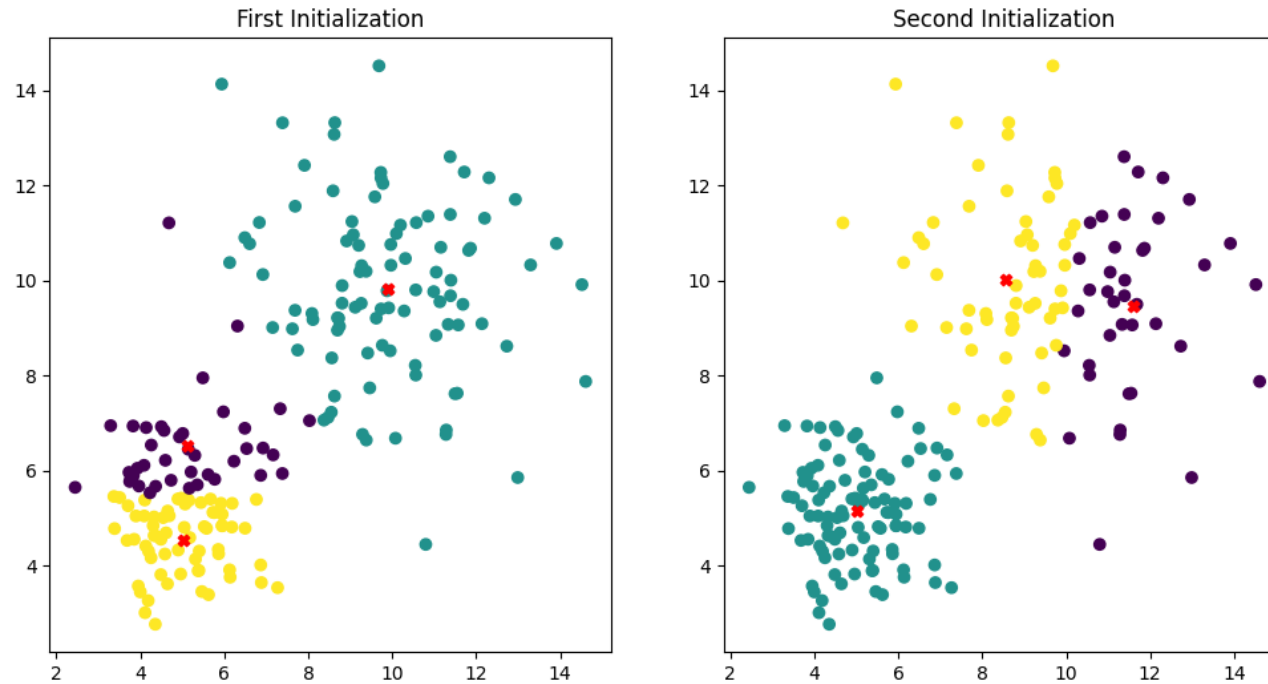
Image from Amruthnath, Nagdev, and Tarun Gupta. "Fault diagnosis using clustering. What statistical test to use for hypothesis testing?." *arXiv preprint arXiv:1904.13365* (2019).

K-means Clustering: initialization

- Question: will k-means reach a global optimum or a local optima?

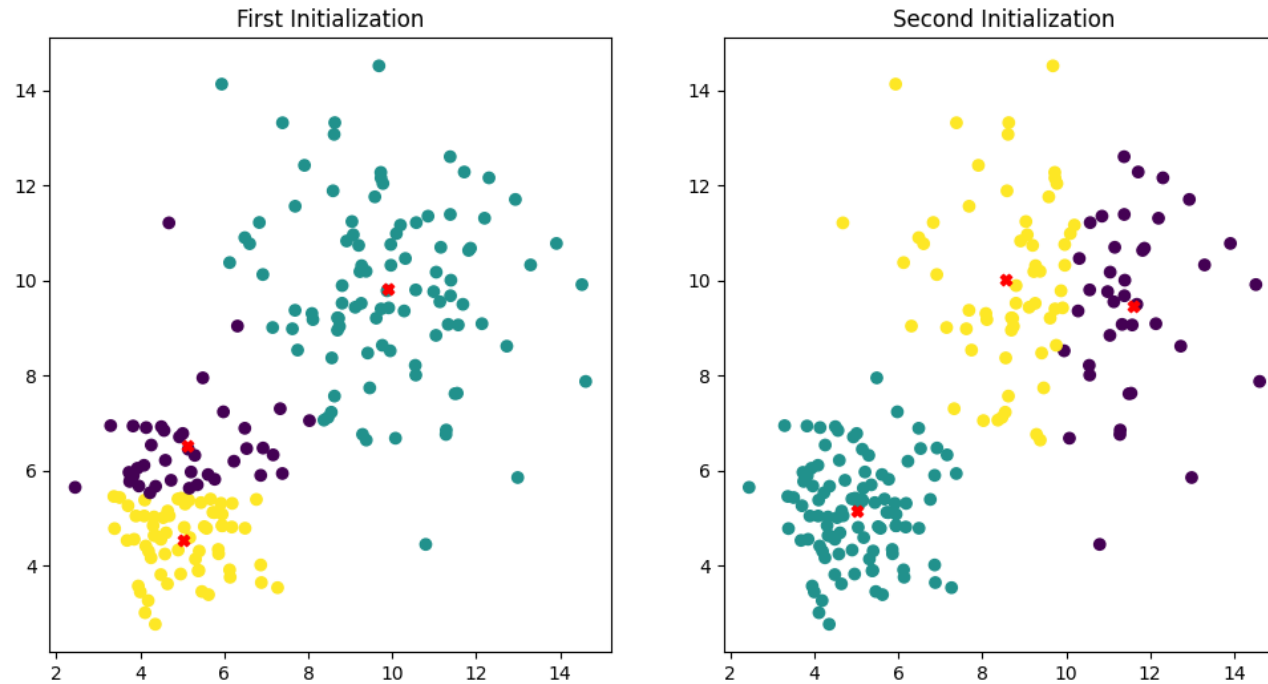
K-means Clustering: initialization

- Question: will k-means reach a global optimum or a local optima?
- Answer: local optima, depending heavily on the initial centroid location



K-means Clustering: initialization

- Question: will k-means reach a global optimum or a local optima?
- Answer: local optima, depending heavily on the initial centroid location
- We'll go through some of the methods you can use to improve performance



K-means Clustering: initialization

- You can run k-means multiple times (**restarts**)
 - Execute k-means m times with different random initializations, and measure the WSS after each one
 - Choose the clustering solution that yielded the lowest WSS
- Pros: easy to implement
- Cons: have to run k-means multiple times, difficulty with reproducibility

K-means Clustering: initialization

- You can use **naïve sharding**
 - Sum the feature values for each sample
 - Sort the samples by this composite value
 - Split the dataset into k equally sized pieces (**shards**)
 - For each shard i , compute the mean $x_{i,\mu}$ (a vector that is the mean of all datapoints)
 - Set $x_{i,\mu}$ as a centroid
- This method attempts to “evenly” space out centroids
- Pros: simple to implement
- Cons: sensitive to data distribution, composite value may not accurately represent spread of data

K-means Clustering: initialization

- You can use k-means++
 1. Choose the first centroid μ_1 from the data
 1. Meaning that μ_1 will fall on a datapoint in X
 2. For each datapoint x_i , compute the $d(x_i)$, the distance between x_i and the closest centroid from the current set of centroids
 3. Choose a new centroid from the remaining datapoints with probability proportional to $d(x_i)^2$
 4. Repeat steps 2 and 3 until k centroids have been chosen
- This method tries to maximize the spread of the centroids
- Pros: improves quality of local minima, faster convergence to local minima
- Cons: computationally more expensive at beginning compared to random initialization

K-means Clustering: initialization

- There are no “right answers” in clustering
- It all depends on domain knowledge and result interpretation
- K-means is trying to optimize the **heterogeneity objective**, the WSS through **coordinate descent**
 - Heterogeneity **always decreases or stays the same** as k-means runs

$$\operatorname{argmin}_{z, \mu} \sum_{i=1}^k \sum_{j=1}^n \mathbf{1}\{z_i = j\} \left\| x_j^{(i)} - \mu_i \right\|^2$$

Given a dataset of n datapoints and hyperparameter k , find clusters z with means μ

Step 0: Initialize cluster centroids randomly

Repeat until convergence:

Step 1: Assign each example to its closest cluster centroid

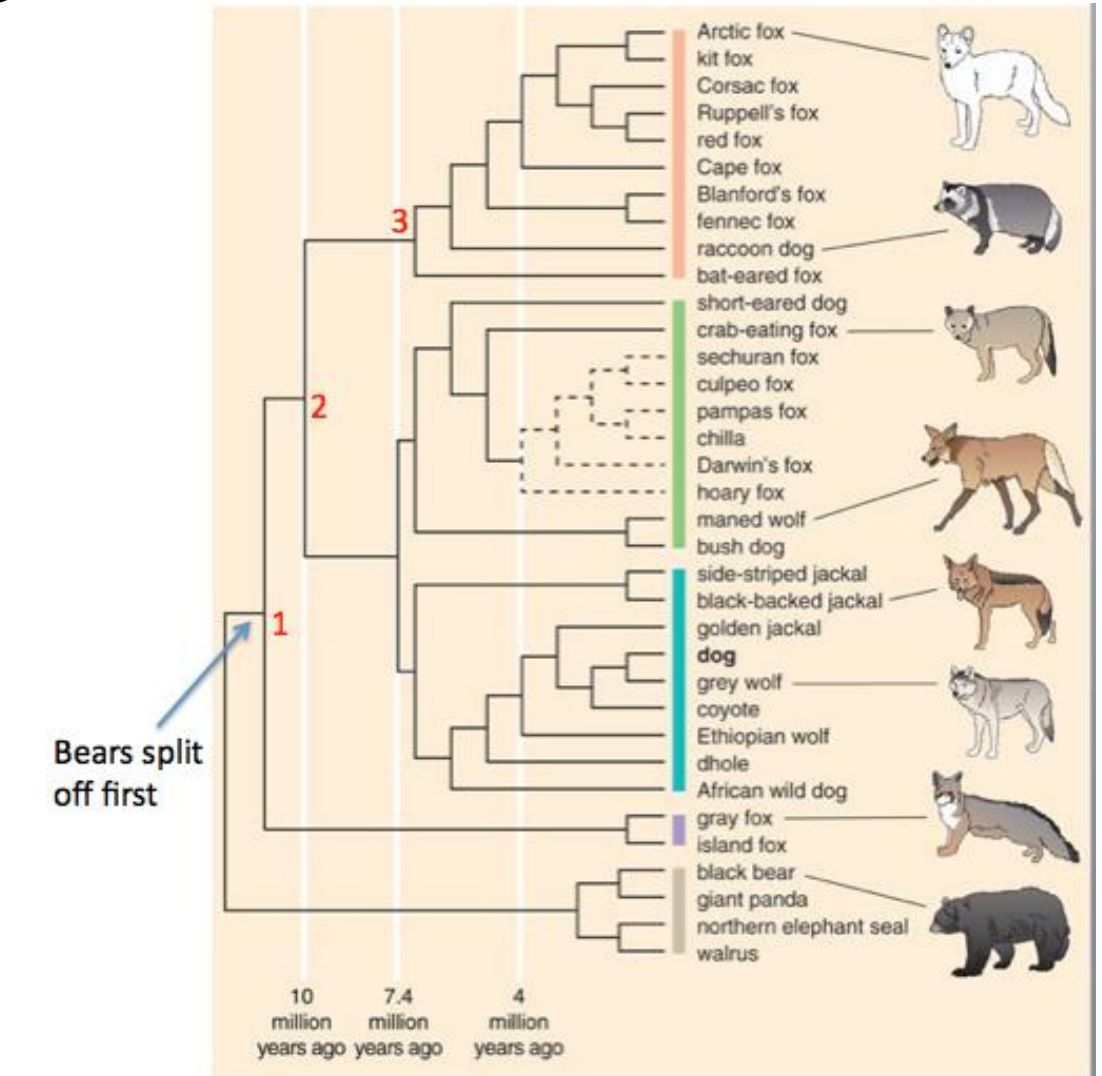
Fix μ , minimize over z

Step 2: Update the centroids to be the average of all the points assigned to that cluster

Fix z , minimize over μ

Hierarchical Clustering

- A lot of data is **hierarchical**
- We can use **hierarchical clustering!**
 - Avoids choosing the # of clusters beforehand
 - Can visualize clusters using **dendrograms**
 - Can use any distance metric (K-means requires Euclidean distance)
 - Can find more complex shapes than K-means

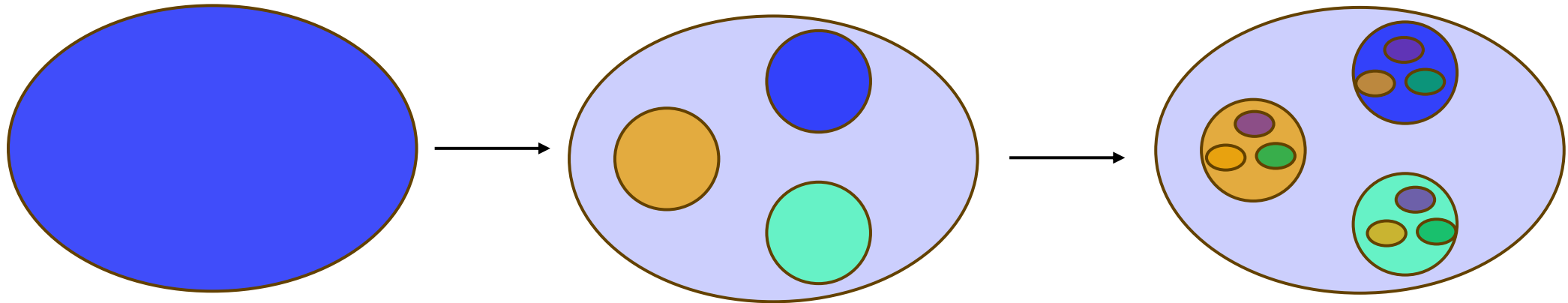


Types of Hierarchical Clustering

- **Divisive** (top-down)
 - Start with data in one (big) cluster
 - Recursively split the data into smaller clusters
- **Agglomerative** (bottom-up)
 - Start with each data point in its own cluster
 - Merge clusters until all points are in one (big) cluster

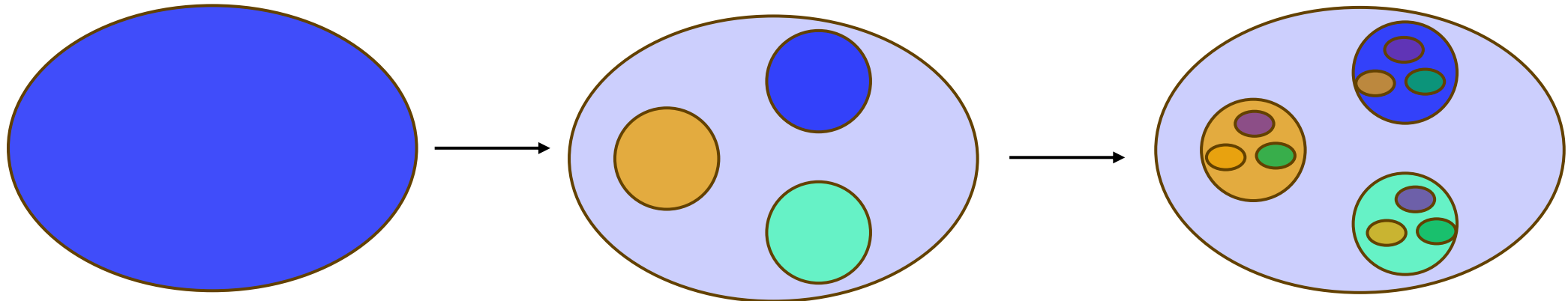
Divisive Clustering

- **Divisive** (top-down)
 - Start with data in one (big) cluster
 - Run k-means to divide the data into smaller clusters
 - Run k-means on each smaller cluster
 - And so on!



Divisive Clustering

- Design choices
 - Which clustering algorithm to use (e.g. k-means)
 - How many clusters per split
 - When to stop splitting
 - Max cluster size (number of points in each cluster)
 - Max cluster radius (distance from centroid to furthest point)
 - Some number of clusters

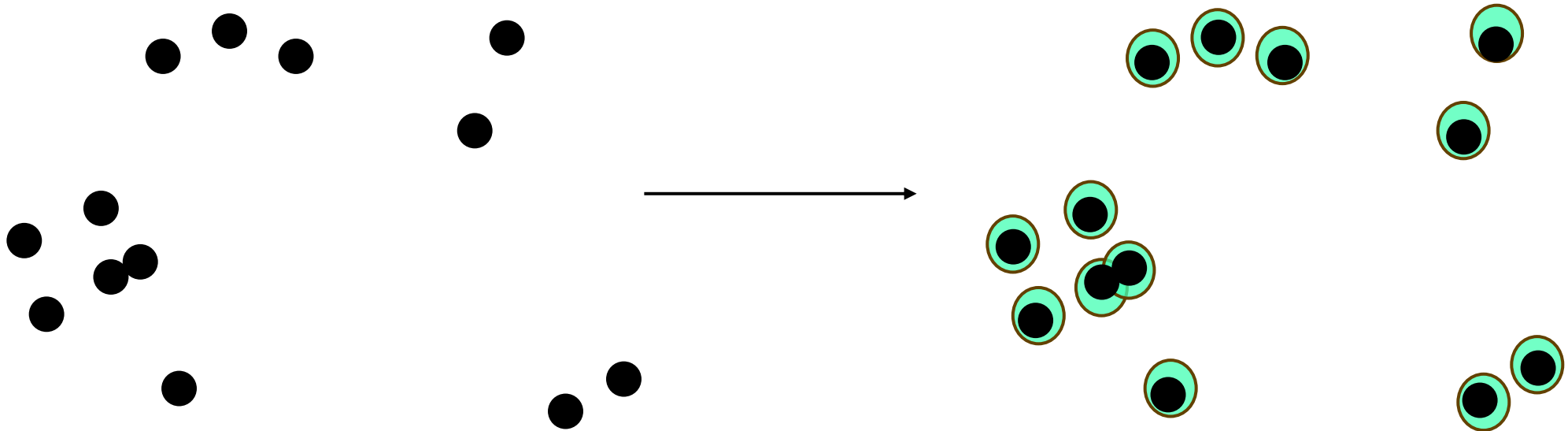


Agglomerative Clustering

- Agglomerative Clustering
 - Initialize each point in its own cluster
 - Choose a distance metric between clusters (hyperparameter)
 - While there is more than one cluster
 - Merge the two closest clusters together

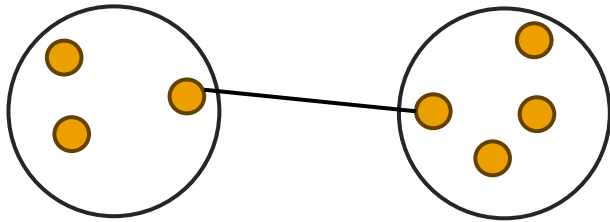
Agglomerative Clustering

- Agglomerative Clustering
 - **Initialize each point in its own cluster**
 - Choose a distance metric between clusters and linkage function (hyperparameters)
 - While there is more than one cluster
 - Merge the two closest clusters together

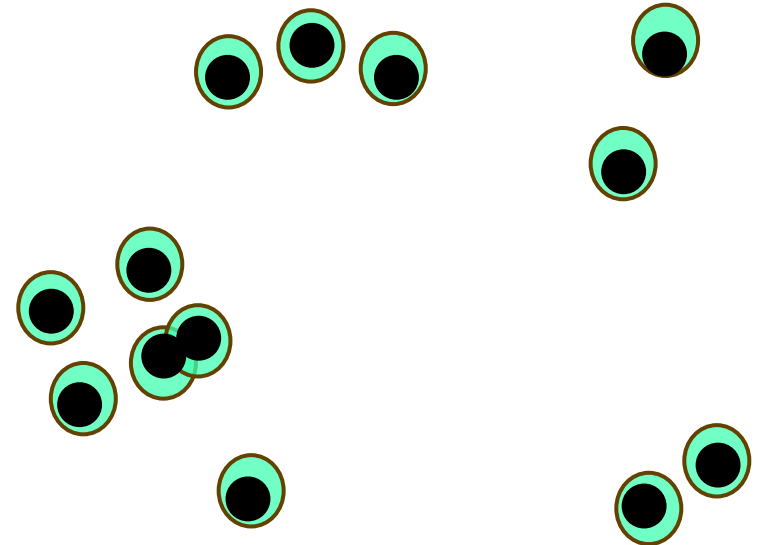


Agglomerative Clustering

- Agglomerative Clustering
 - Initialize each point in its own cluster
 - **Choose a distance metric between clusters and linkage function (hyperparameters)**
 - While there is more than one cluster
 - Merge the two closest clusters together

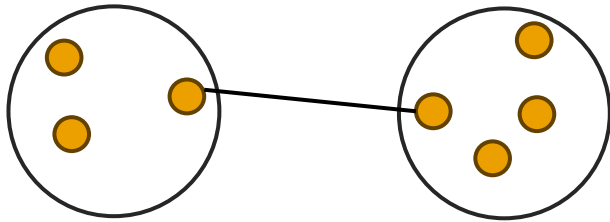


- Euclidean distance
- Single Linkage: $dist(c_1, c_2) = \min_{x_i \in c_1, x_j \in c_2} d(x_i, x_j)$
- This defines distance between two clusters as the smallest distance between any pair of points between the clusters

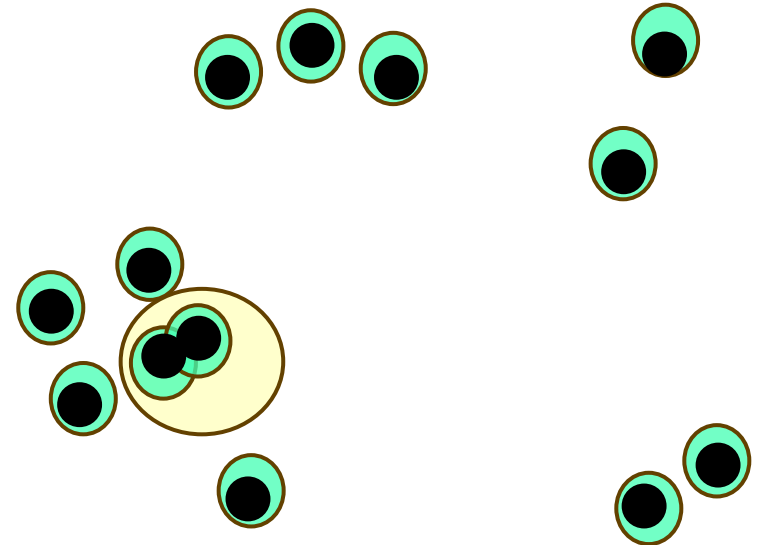


Agglomerative Clustering

- Agglomerative Clustering
 - Initialize each point in its own cluster
 - Choose a distance metric between clusters and linkage function (hyperparameters)
 - **While there is more than one cluster**
 - **Merge the two closest clusters together**

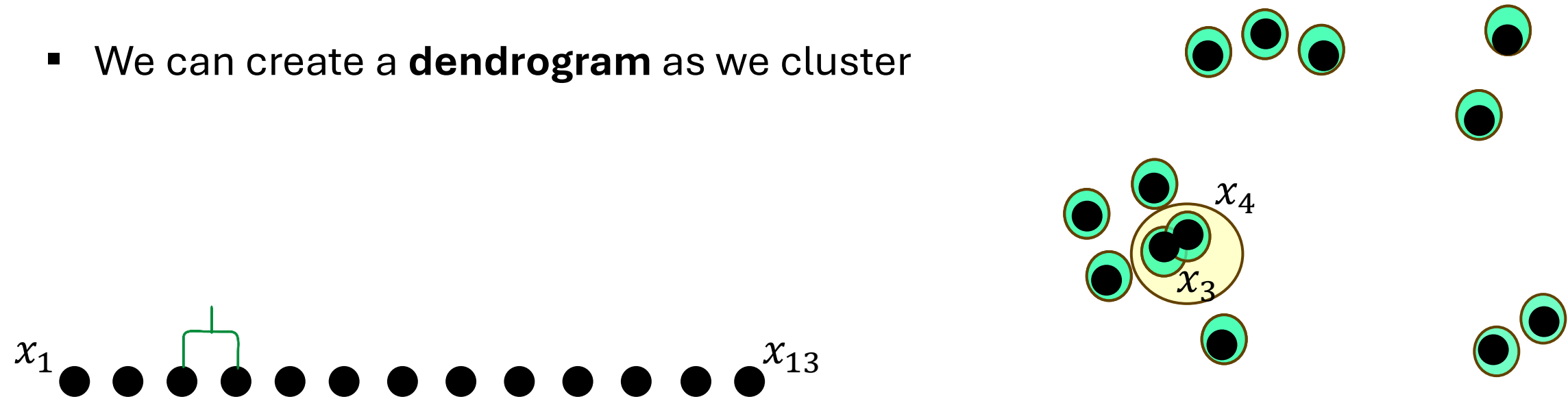


- Euclidean distance
- Single Linkage: $dist(c_1, c_2) = \min_{x_i \in c_1, x_j \in c_2} d(x_i, x_j)$
- This defines distance between two clusters as the smallest distance between any pair of points between the clusters



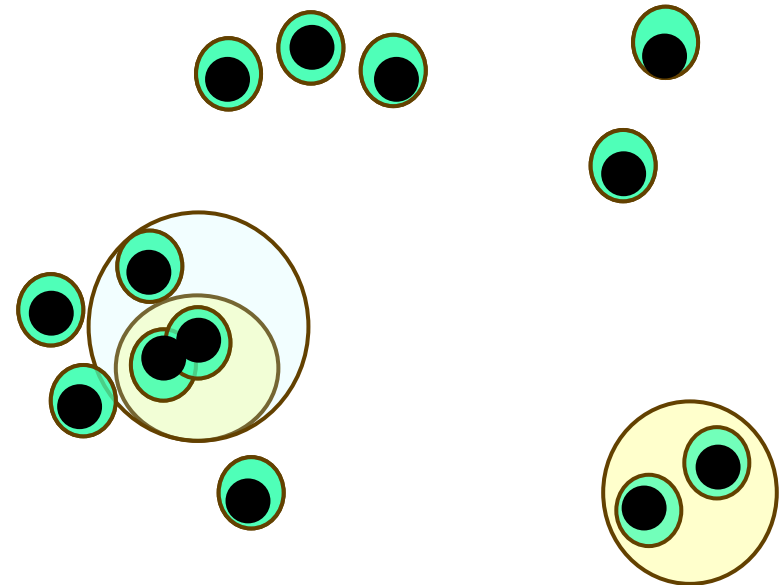
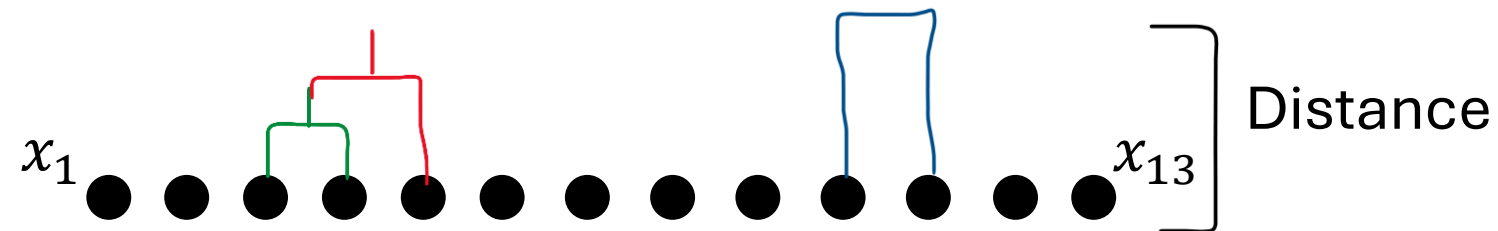
Agglomerative Clustering

- Agglomerative Clustering
 - Initialize each point in its own cluster
 - Choose a distance metric between clusters and linkage function (hyperparameters)
 - **While there is more than one cluster**
 - **Merge the two closest clusters together**
- We can create a **dendrogram** as we cluster



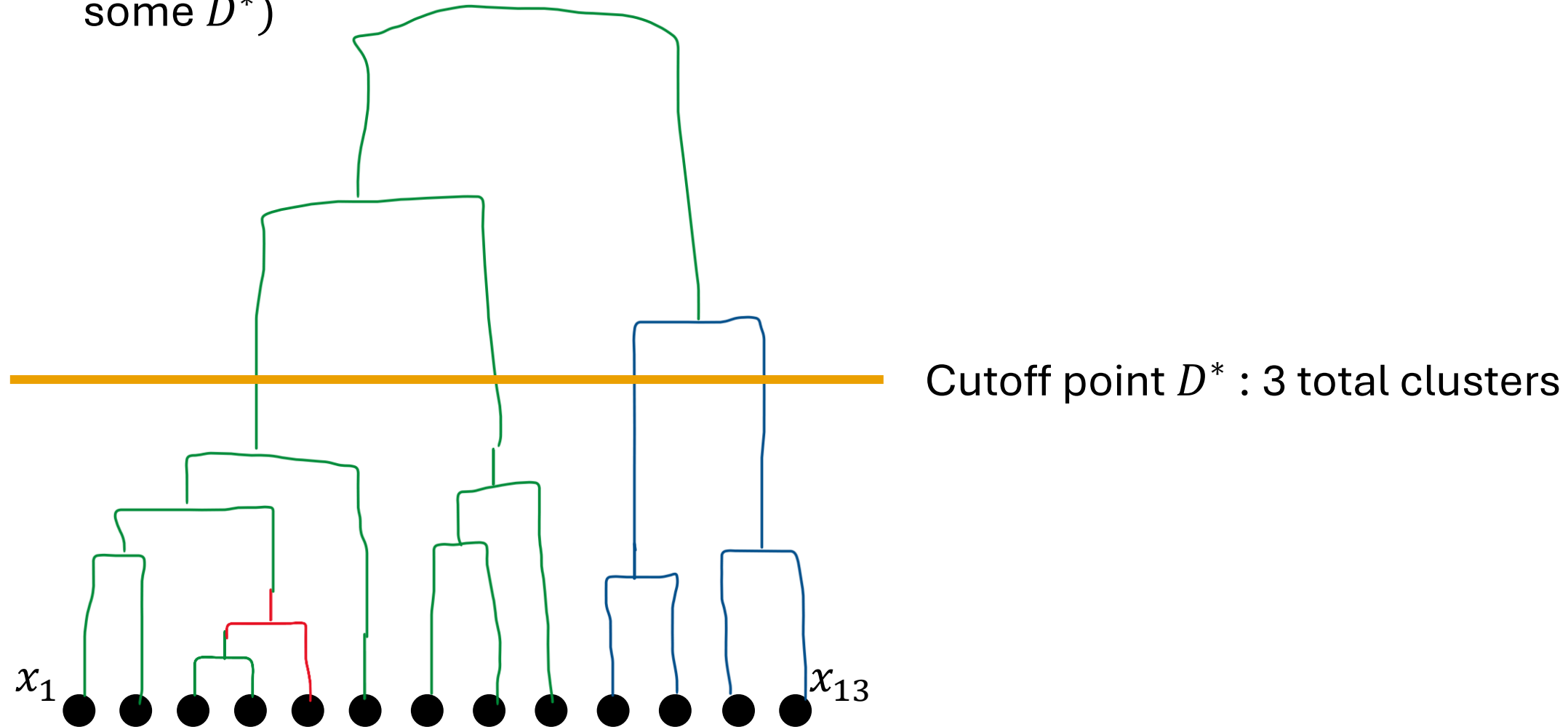
Agglomerative Clustering

- Agglomerative Clustering
 - Initialize each point in its own cluster
 - Choose a distance metric between clusters and linkage function (hyperparameters)
 - **While there is more than one cluster**
 - **Merge the two closest clusters together**
- We can create a **dendrogram** as we cluster
- The height of the dendrogram grows as we group points farther from each other



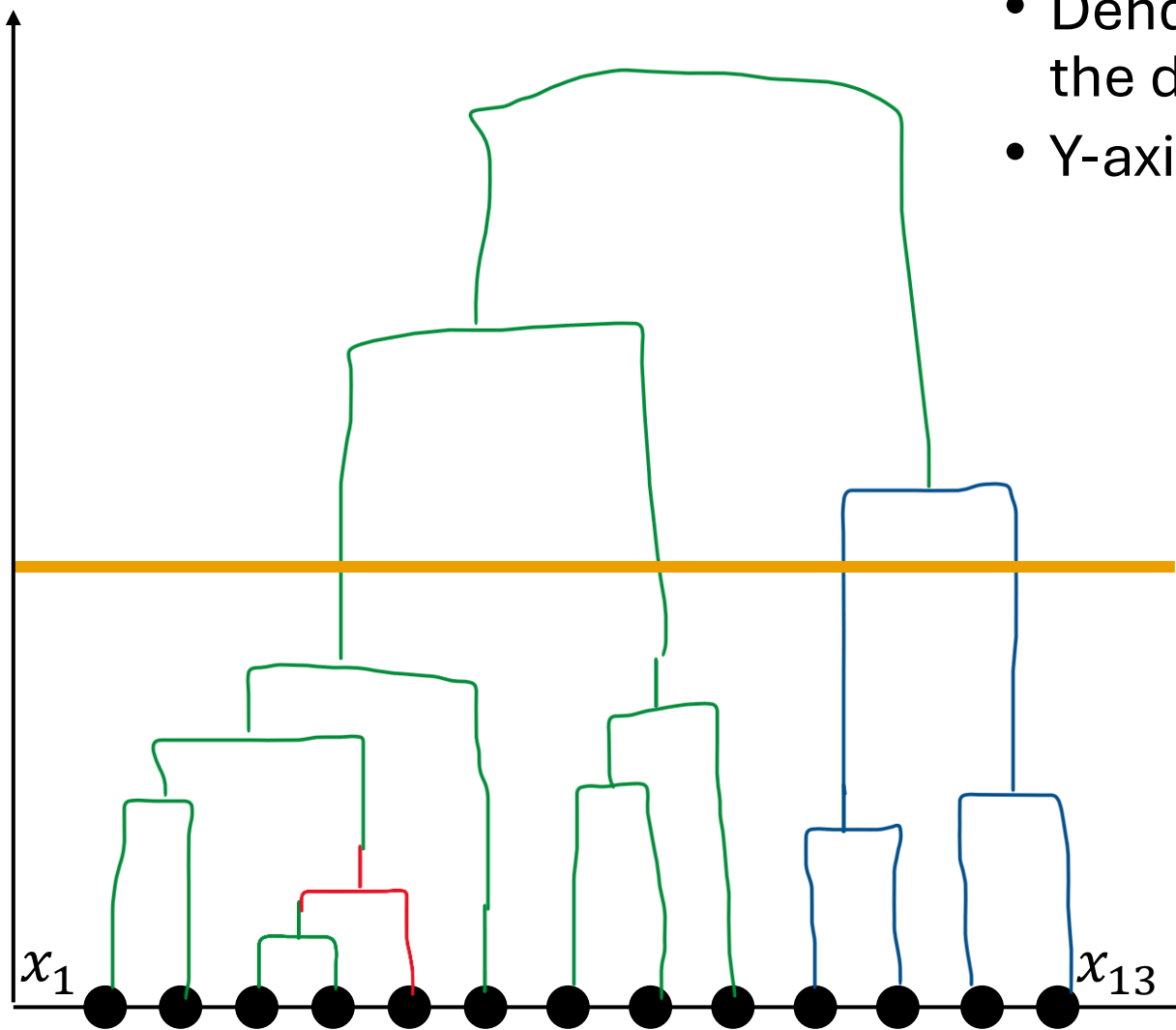
Agglomerative Clustering

- We can cut off the merging when we start merging clusters with a large distance (over some D^*)



Agglomerative Clustering

Cluster distance



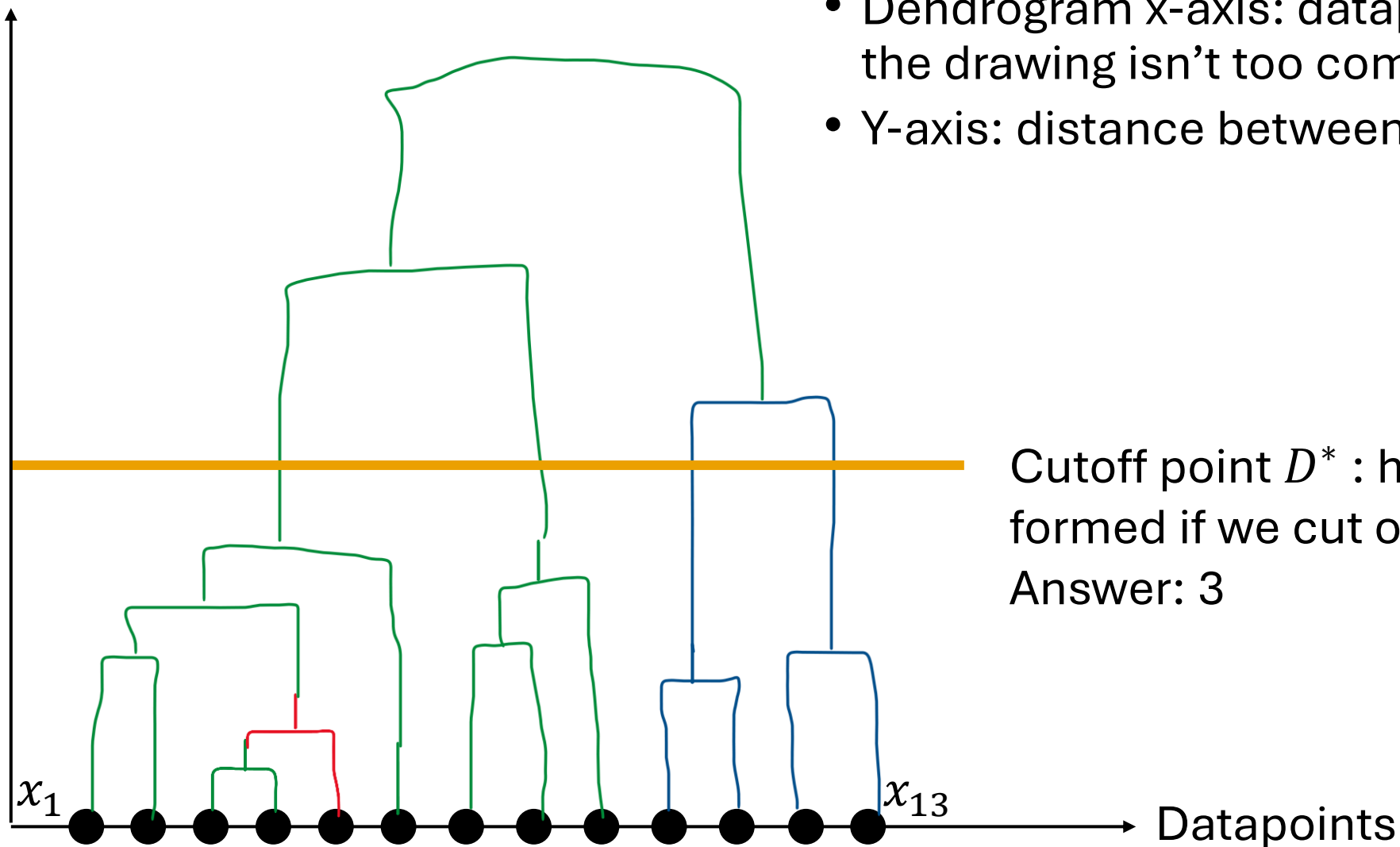
- Dendrogram x-axis: datapoints (arranged so that the drawing isn't too complex)
- Y-axis: distance between merged clusters

Cutoff point D^* : how many clusters are formed if we cut off here?

Datapoints

Agglomerative Clustering

Cluster distance



- Dendrogram x-axis: datapoints (arranged so that the drawing isn't too complex)
- Y-axis: distance between merged clusters

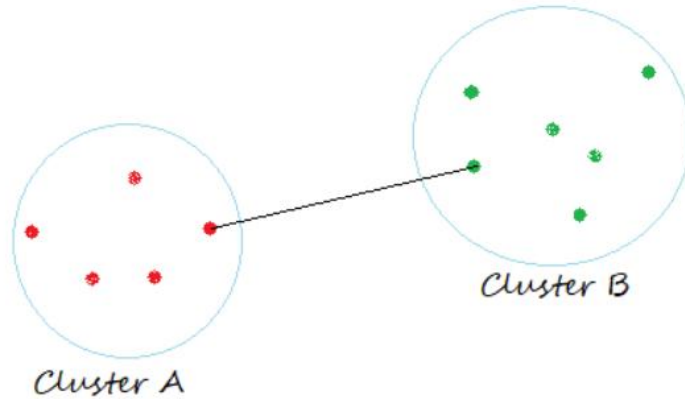
Cutoff point D^* : how many clusters are formed if we cut off here?
Answer: 3

Agglomerative Clustering

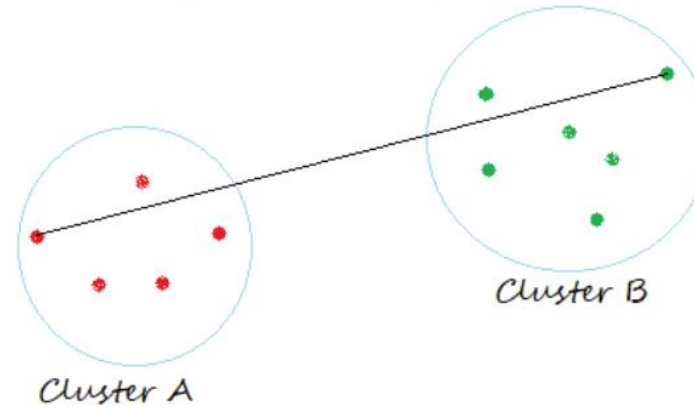
- Design choices
 - Distance metric (Euclidean, Manhattan, etc)
 - Linkage function
 - Single linkage: $D(C_1, C_2) = \min_{x_i \in C_1, x_j \in C_2} d(x_i, x_j)$
 - Complete linkage: $D(C_1, C_2) = \max_{x_i \in C_1, x_j \in C_2} d(x_i, x_j)$
 - Centroid linkage: $D(C_1, C_2) = d(\mu_1, \mu_2)$
 - Others

Agglomerative Clustering

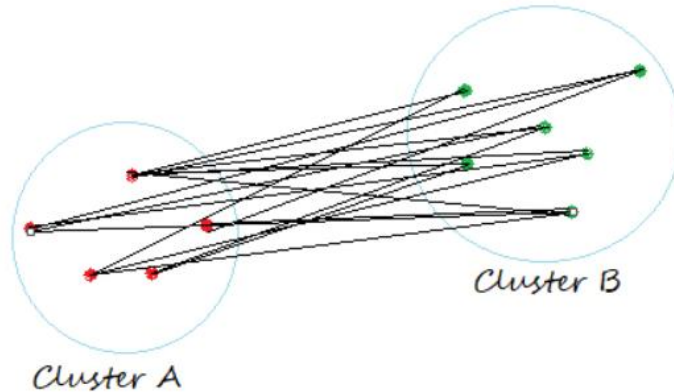
Single Linkage



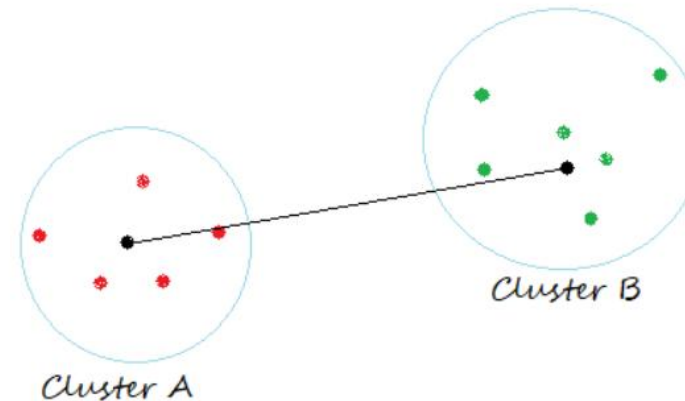
Complete Linkage



Average Linkage



Centroid Linkage



Break

Get some water, stretch your legs, chat, be back in 10

Hard Clustering

- These clustering algorithms associate each point with one (and only one) cluster
- What if we want probabilities of each point belonging to all of the clusters, like with classification?
- We can use a different algorithm, Gaussian Mixture Models (GMMs)

Gaussian Mixture Model

- Gaussian Mixture Models (GMMs) take a probabilistic approach to clustering
- A Gaussian Mixture is a function composed of several Gaussians (in our case, K)
- Each Gaussian is comprised of:
 - A mean μ (the center)
 - A covariance matrix Σ
 - In 1D, this is just the variance/width of the Gaussian
 - With more features, it is a square matrix where the diagonal elements are the variance/width of the gaussian for each feature, and the other elements are the correlation/covariance between pairs of features
 - A mixing probability π (the probability that a given data point was generated from this Gaussian, effectively the “size” of the Gaussian)
 - The mixing probabilities for all Gaussians must sum to 1: $\sum_{i=1}^K \pi_i = 1$

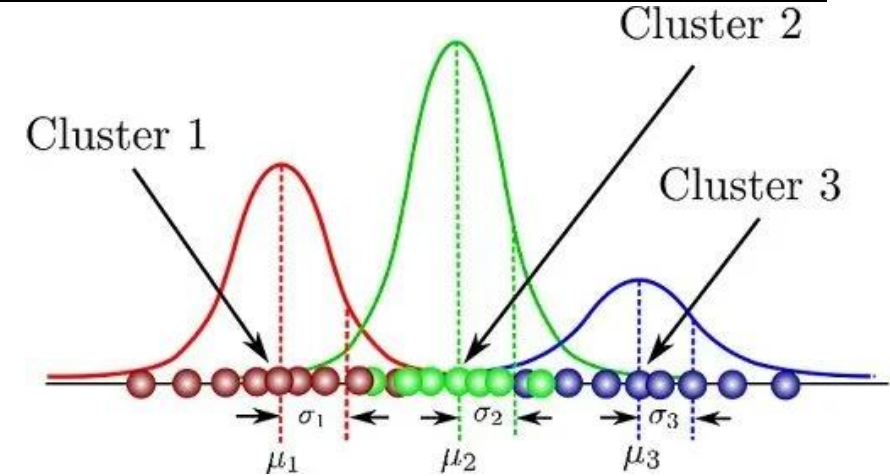
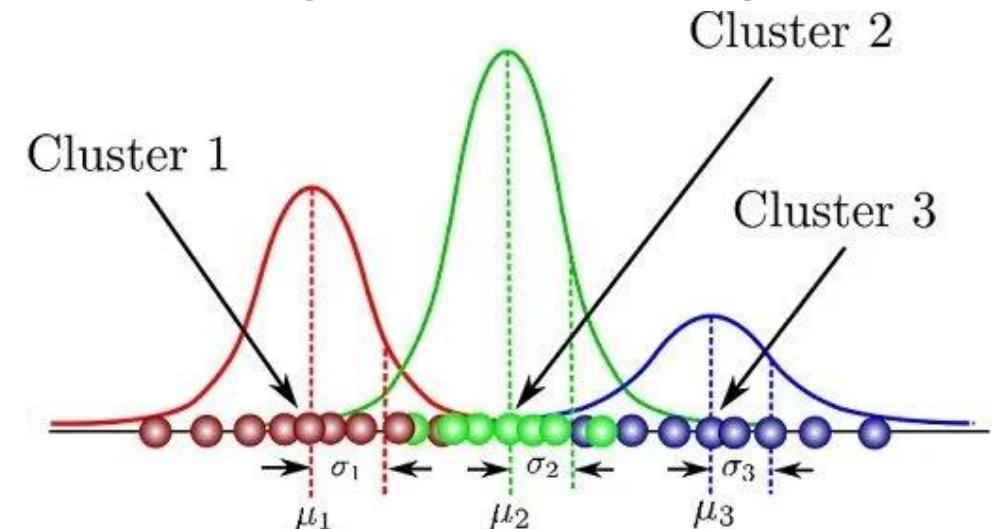


Image from <https://builtin.com/articles/gaussian-mixture-model>

Gaussian Mixture Model

- To find the optimal values for each parameter, we want to find a set of K Gaussians that maximize the likelihood of our dataset
- To start, we consider $P(x)$, where x is a d -dimensional vector (a datapoint)
 - What is $P(x)$?
 - What if instead of being given a dataset, we generated one based on Gaussian distributions
 - The image shows a 1D dataset, which could have been generated through sampling from the three Gaussians shown

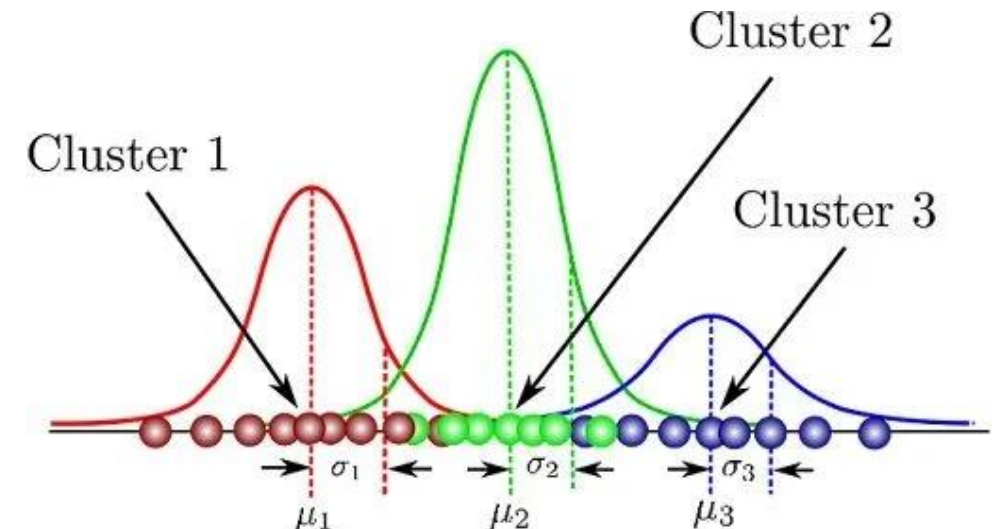


Gaussian Mixture Model

- To find the optimal values for each parameter, we want to find a set of K Gaussians that maximize the likelihood of our dataset
- The Gaussian density function (the likelihood of the Gaussian producing a datapoint $\mathbf{x}$ is given by the following equation:

$$\mathcal{N}(\mathbf{x}|\mu, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$$

- Where D is the number of features
- μ is the mean
- Σ is the covariance matrix
- π is just pi here (3.14), not the mixing coefficient



Gaussian Mixture Model

- To find the optimal values for each parameter, we want to find a set of K Gaussians that maximize the likelihood of our dataset
- Let z_{nk} be a variable that is 1 when x_n comes from Gaussian k
- We want to find $P(z_{nk} = 1|x_n)$: given a datapoint x_n , what is the probability it came from Gaussian k ?
- Some helpful pieces of information:
 - $P(z_k = 1) = \pi_k$: the probability of observing a datapoint from Gaussian k is the mixing coefficient π_k
 - $P(\mathbf{x}_n|z_k) = \mathcal{N}(\mathbf{x}_n|\mu_k, \Sigma_k)$: the probability of observing datapoint x_n given the current definition for Gaussian k is just the Gaussian density function

Gaussian Mixture Model

- Using the product rule of probabilities ($P(A, B) = P(A|B)P(B)$), we can find that $P(\mathbf{x}_n)$ (the probability of seeing a datapoint $\mathbf{x}_n$ in our dataset) is

$$P(\mathbf{x}_n) = \sum_{k=1}^K P(\mathbf{x}_n, \mathbf{z}) = \sum_{k=1}^K P(\mathbf{x}_n|\mathbf{z})P(\mathbf{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n|\mu_k, \Sigma_k)$$

- The likelihood of the entire dataset $P(\mathbf{X})$ can now be defined as the joint probability of all of our datapoints

$$P(\mathbf{X}) = \prod_{n=1}^N P(\mathbf{x}_n) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n|\mu_k, \Sigma_k)$$

- Just like in Logistic Regression, we want to convert products to summations for easier computation, so we'll use log-likelihood

$$\ln(P(\mathbf{X})) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n|\mu_k, \Sigma_k) \right)$$

Gaussian Mixture Model

- Unfortunately, we can't apply **MLE** (Maximum Likelihood Estimation) like we did for logistic regression
 - The function is complex, non-linear, and the mixture weights are not observable from the data
- Fortunately, we can use the **Expectation-Maximization** (EM) algorithm instead!
- We start by randomly initializing model parameters
 - Can be done by randomly choosing k centroids (or running k-means to choose initial centroids), and then computing GMM parameters based on centroids
- Then repeat the Expectation and Maximization steps until convergence:
 - Expectation: Compute the expected log-likelihood of the model
 - Maximization: Update the parameters of the model to maximize the log-likelihood of the observed data
- Convergence: often a maximum number of iterations or when the change in log-likelihood is minimal (under some ϵ)

Gaussian Mixture Model

- We'll cover the rest of expectation maximization next week; I'll assign some reading to do before class next week so you can get more familiar with the math
- Before moving on, we'll cover how we can calculate the probability of each z_k **given** X (used for cluster probabilities)
- We know the probability of seeing any $\mathbf{x}_n$

$$P(\mathbf{x}_n) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- The probability of seeing $\mathbf{x}_n$ given z_k

$$P(\mathbf{x}_n | z_k) = \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- And the probability of observing a datapoint from Gaussian z_k

$$P(z_k = 1) = \pi_k$$

- What we really want to know is the probability of each z_k **given** $\mathbf{x}_n$

Probability Aside: Bayes' Rule

- When we have two events, A and B , we might want to calculate the probability of A happening given that B has happened.
- Example: consider a rare disease with a test that can have false positives and negatives. The disease is present in 1% of the population. If someone has the disease, the test will give a **true positive** result 99% of the time. The overall probability of testing positive (true positives and false positives) is 2%.
- Let A be the event in which the person has the disease, and B be a positive test result.
- So $P(A) = 0.01$, $P(B|A) = 0.99$, and $P(B) = 0.02$
- Question: what is the probability that a person has the disease given that they test positive?
 - Or, what is $P(A|B)$?

Probability Aside: Bayes' Rule

- Question: what is the probability that a person has the disease given that they test positive?
 - Or, what is $P(A|B)$?
- $P(A) = 0.01$, $P(B|A) = 0.99$, and $P(B) = 0.02$
- Bayes' rule says

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- So even though we don't know $P(A|B)$, since we know $P(B|A)$ we can calculate it!
- $P(A|B) = \frac{0.99*0.01}{0.02} = 0.495$

Gaussian Mixture Model

- Bayes' rule

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- We know

- $P(\mathbf{x}_n) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$
- $P(\mathbf{x}_n | z_k) = \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$
- $P(z_k = 1) = \pi_k$

- So to calculate $P(z_k = 1 | \mathbf{x}_n)$:

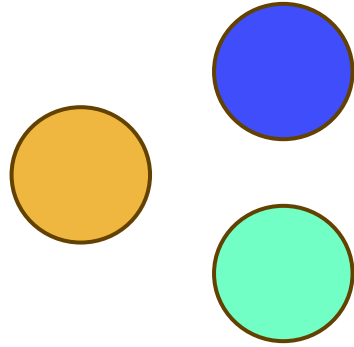
$$P(z_k = 1 | \mathbf{x}_n) = \frac{P(\mathbf{x}_n | z_k = 1)P(z_k = 1)}{P(\mathbf{x}_n)} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}$$

Gaussian Mixture Model

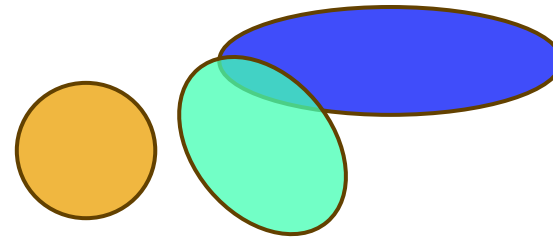
- Pros and Cons of GMMs
 - Pros
 - Can handle elliptical and irregular cluster shapes
 - Can handle clusters of varying sizes
 - Can give probabilities, and allows for overlapping clusters
 - Cons
 - High computational cost
 - Sensitive to initialization
 - Can be mitigated through **restarts**, just like K-means
 - Assumes data is a mixture of Gaussians

Clustering Shapes

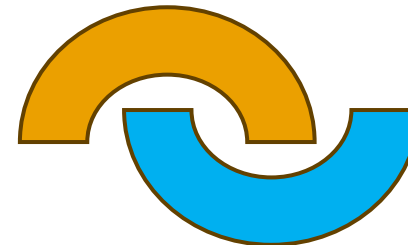
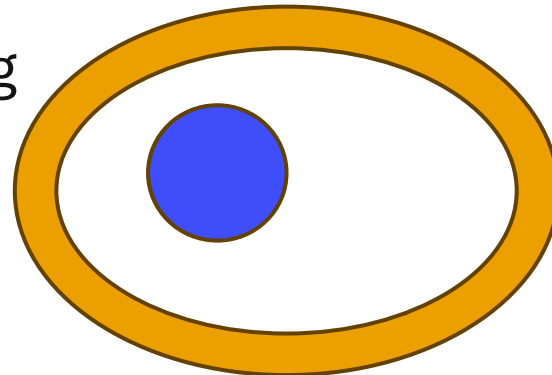
- K-means



- GMMs

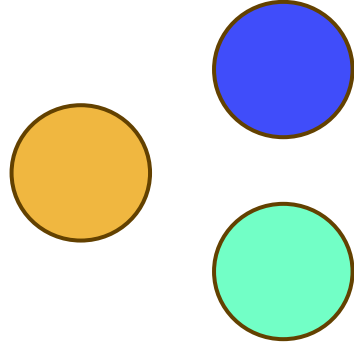


- Agglomerative Clustering

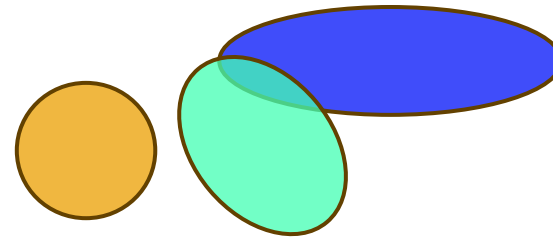


Clustering Shapes

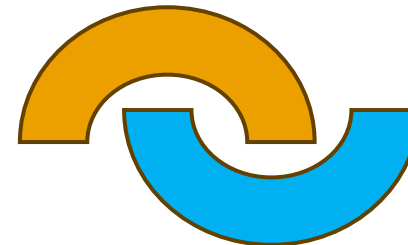
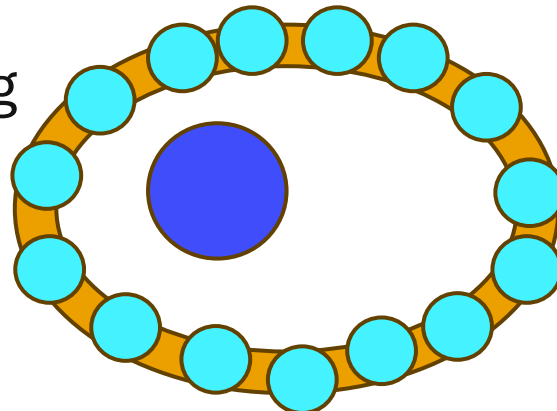
- K-means



- GMMs



- Agglomerative Clustering
 - Single linkage is particularly good at “lines” of datapoints



Clustering Algorithms Compared

- Agglomerative vs Divisive
 - Both are good, but divisive clustering can be more computationally expensive
- Agglomerative clustering
 - No need to choose k beforehand
 - Provides a visualization of cluster relationships (dendrogram)
 - Computationally expensive
 - Complex cluster shapes
 - Cutoff point can be subjective
- K-means
 - Easy to implement
 - Efficient
 - Sensitive to initial centroids
 - Have to choose k in advance
 - Only same-size spherical clusters
- GMMs
 - Provides probability estimates
 - Computationally more expensive than k-means (less than hierarchical)
 - Have to choose k in advance
 - Sensitive to initial parameters
 - Assumes Gaussian distributions